



**TECNOLÓGICO
NACIONAL DE MÉXICO**

INSTITUTO TECNOLÓGICO SUPERIOR DE JEREZ

Carrera: Ingeniería en Sistemas Computacionales

Nombre de la Materia: Tópicos Selectos Web

**Nombre de la Actividad: Reporte de Practica (CACHE
API)**

Nombre del Alumno(a): Nayeli Morales García

Número de Control: S19070105

Semestre: 14

Nombre del Docente: Salvador Acevedo Sandoval

Fecha de Entrega: miércoles 13 de mayo del 2026

Introducción

En esta práctica se trabajó con la implementación de la Cache API mediante el uso de Service Workers en una aplicación web. El objetivo principal fue comprender cómo almacenar recursos en caché para mejorar el rendimiento y permitir el funcionamiento de la aplicación incluso sin conexión a internet. Para ello, se utilizó un entorno con Node.js y Express, creando una estructura básica de proyecto que incluye archivos como app.js, sw.js y server.js. A lo largo del desarrollo, se exploró el proceso de registro del Service Worker, la gestión del almacenamiento en caché y la interacción con datos obtenidos desde una API.

Desarrollo

Primero creamos la carpeta de la práctica.



Entramos en la terminal del editor y entramos a la carpeta que creamos desde ahí.

```
● PS C:\Users\Nayeli\Desktop\Topicos Programacion WEB\Talleres> cd CacheAPI
○ PS C:\Users\Nayeli\Desktop\Topicos Programacion WEB\Talleres\CacheAPI> |
```

Ahora inicializamos el proyecto con npm init -y y luego de eso instalamos express que lo vamos a utilizar con npm install express.

```
● PS C:\Users\Nayeli\Desktop\Topicos Programacion WEB\Talleres\CacheAPI> npm init -y
Wrote to C:\Users\Nayeli\Desktop\Topicos Programacion WEB\Talleres\CacheAPI\package.json:

{
  "name": "cacheapi",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

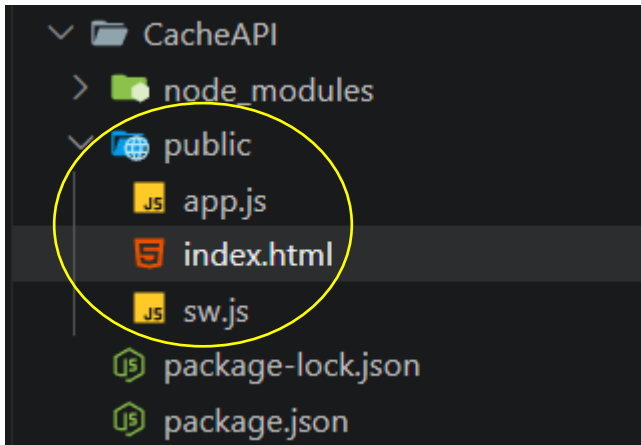
● PS C:\Users\Nayeli\Desktop\Topicos Programacion WEB\Talleres\CacheAPI> npm install express

added 66 packages, and audited 67 packages in 5s

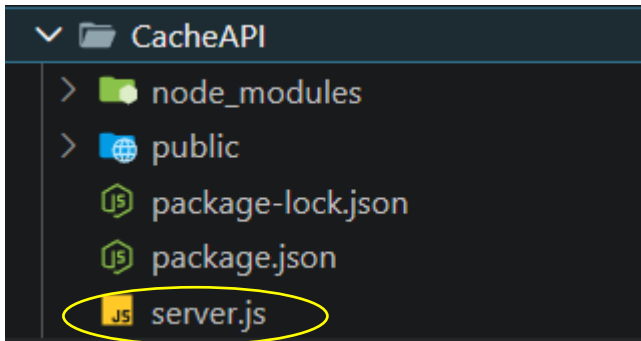
24 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
○ PS C:\Users\Nayeli\Desktop\Topicos Programacion WEB\Talleres\CacheAPI> |
```

Luego de esto, en la carpeta del proyecto, creamos una carpeta llamada “public”. Dentro de ella creamos los archivos: app.js, index.html, sw.js.



Despues en la carpeta raíz del proyecto, creamos el archivo server.js.



Abrimos el archivo server.js y agregamos el siguiente código que es para utilizar express y ubicar la carpeta donde están los archivos a utilizar que en este caso es public y también indicar el puerto y la ruta donde va a funcionar el proyecto.

```
server.js  X
CacheAPI > JS server.js > ...
1  const express = require('express');
2  const path = require('path');
3
4  const app = express();
5
6  const PORT = 3000;
7
8  app.use(express.static('public'));
9
10 app.listen(PORT, () => {
11   console.log(`Servidor ejecutandose en http://localhost:${PORT}`);
12 })
```

Ahora abrimos el archivo app.js y agregamos código. Este código se encarga de ver si el navegador soporta service worker, si esta bien mandara el mensaje de “Service Worker Registrado” si no funciona pues mandara el error. También espera el click en el botón que colocamos en el index, hace una petición a una API luego convierte la petición en json y muestra los datos en la página.

```
app.js
CacheAPI > public > app.js > ...
1  if ('serviceWorker' in navigator){
2      navigator.serviceWorker.register('sw.js')
3      .then(() => {
4          console.log('Service Worker Registrado');
5      })
6      .catch(error => {
7          console.log('Error:', error);
8      });
9  }
10
11
12  document.getElementById('btn').addEventListener('click', () =>{
13      fetch('http://jsonplaceholder.typecode.com/post/1')
14      .then(res => res.json())
15      .then(data =>{
16          document.getElementById('resultado').innerHTML = `
17              <h3>${data.title}</h3>
18              <p>${data.body}</p>
19              `;
20      });
21  });
```

Ahora abrimos el archivo `sw.js` y agregamos código. Este código lo que hace es que el Service Worker implementa una estrategia básica de “cache primero, red después”, donde intenta servir los archivos desde el almacenamiento local antes de ir a buscarlos al servidor. Sin embargo, el código tiene errores importantes, como el uso incorrecto de `require`, la inconsistencia entre `CACHE_HOME` y `CACHE_NAME`, y posibles fallos si algún recurso no se puede cachear.

```
sw.js
CacheAPI > public > sw.js > ...
1  const { event } = require("jquery");
2
3  const CACHE_HOME = 'mi-cache-v3';
4
5  const urlsToCache = [
6    '/',
7    'index.html',
8    'app.js'
9  ];
10
11 self.addEventListener('install', event => {
12   event.waitUntil(
13     caches.open(CACHE_NAME)
14       .then(cache =>{
15         return cache.addAll(urlsToCache);
16       })
17   );
18 });
19
20 self.addEventListener('fetch', event => {
21   event.respondWith(
22     caches.match(event.request)
23       .then(response =>{
24         if(response){
25           return response;
26         }
27         return fetch(event.request);
28       })
29   );
30 });
```

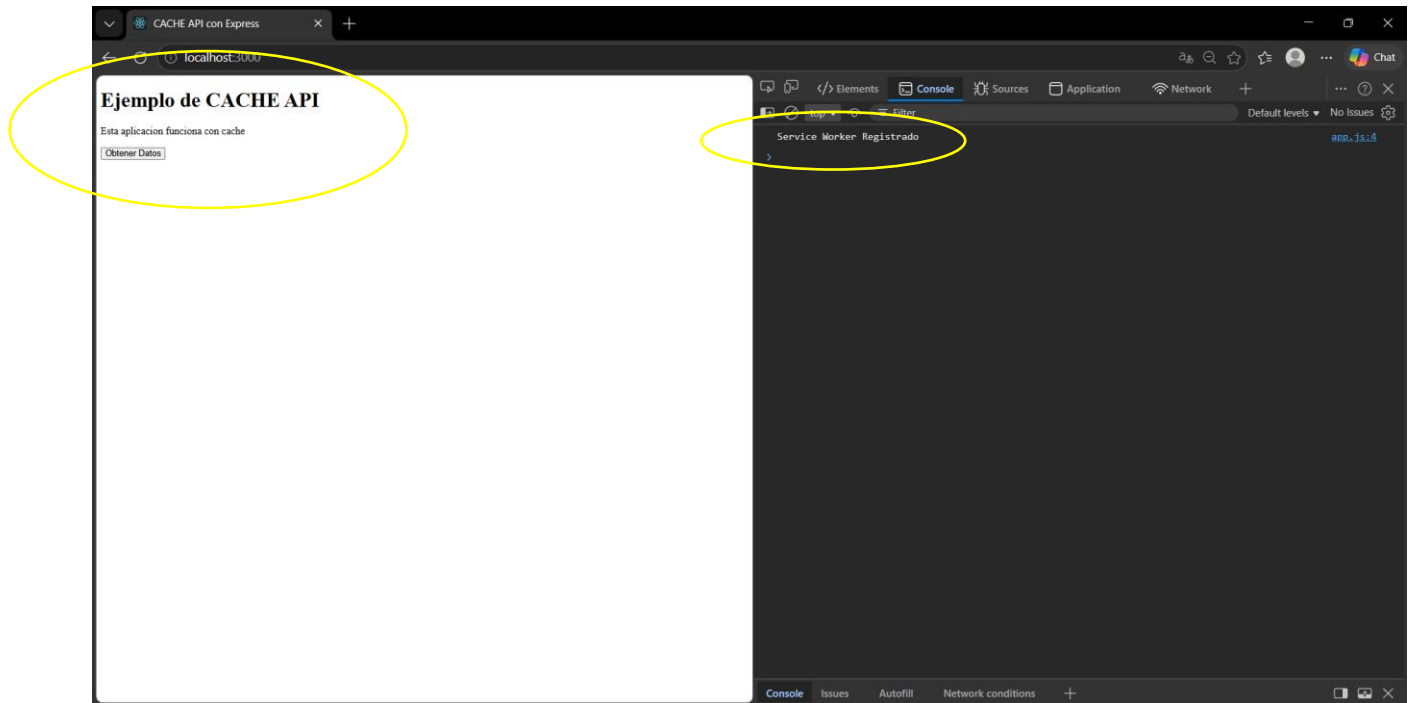
Aquí en index agregamos el formato de HTML 5, agregamos un título para la pestaña de la página y un título para lo que será el espacio de la página. También se agrega un botón para obtener los datos se manda llamar a los resultados y en el script se hace referencia a la app.js que es donde esta el funcionamiento del service worker.

```
index.html X
CacheAPI > public > index.html > html > body > script
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>CACHE API con Express</title>
7  </head>
8  <body>
9
10     <h1> Ejemplo de CACHE API</h1>
11     <p>Esta aplicacion funciona con cache</p>
12     <button id="btn"> Obtener Datos </button>
13     <div id="resultado"> </div>
14     <script src="app.js"> </script>
15
16 </body>
17 </html>
```

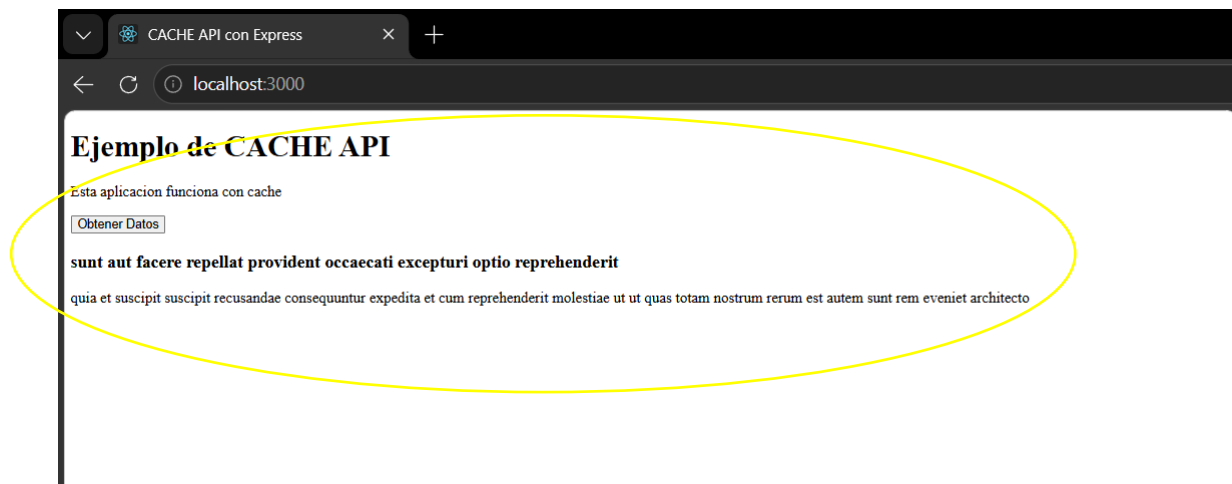
Ya teniendo todo el código de los archivos anteriores ahí que correr el servidor con el comando node server.js que es donde configuramos para que arranque el servidor.

```
PS C:\Users\Nayeli\Desktop\Topicos Programacion WEB\Talleres\CacheAPI> node server.js
Servidor ejecutandose en http://localhost:3000
█
```

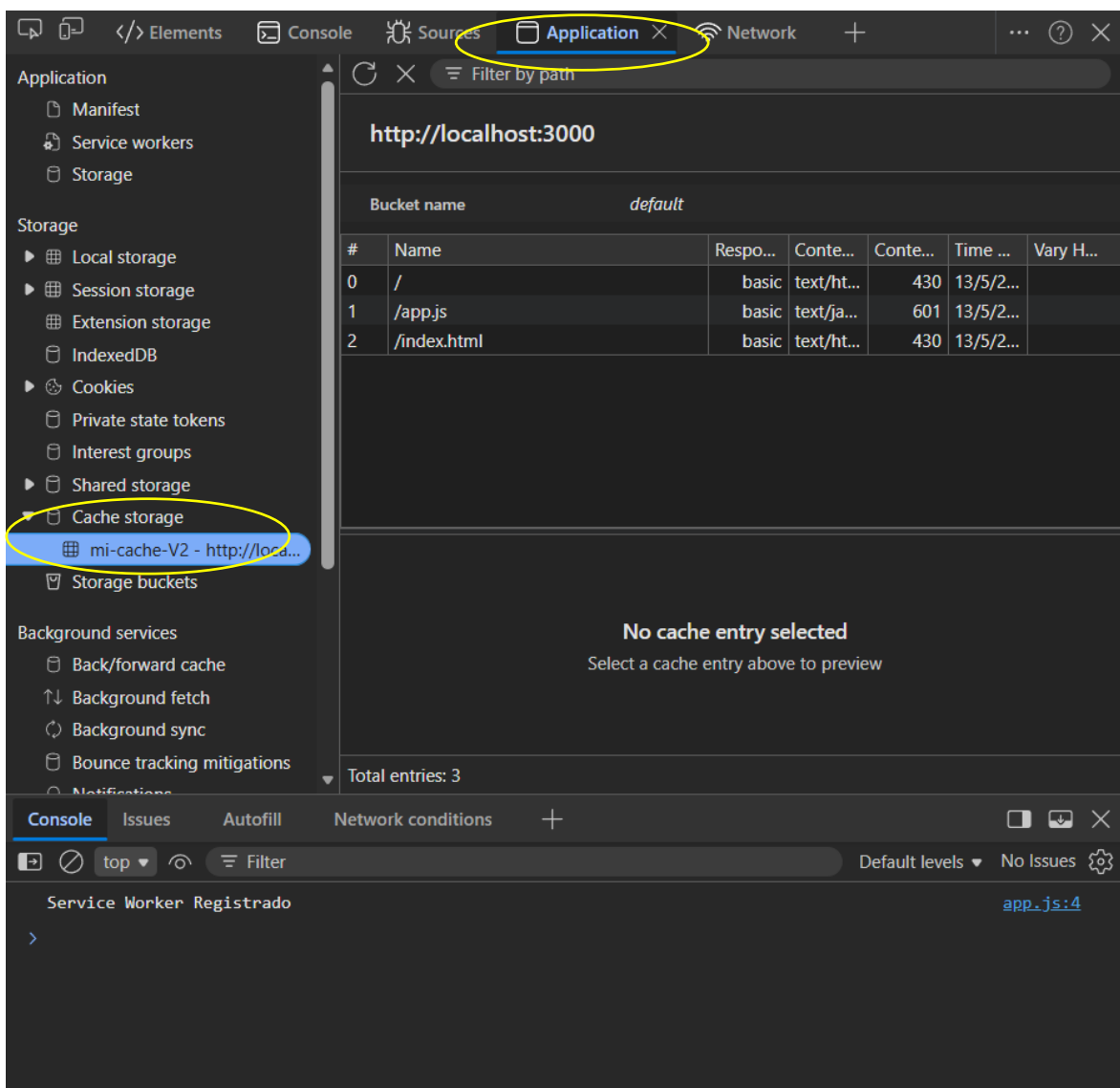
Entramos a la ruta que nos aparece y pues nos muestra el index que esta en la parte blanca de la pantalla y abrimos el inspeccionar y podemos ver que nos da el mensaje que el service worker esta registrado. Esto quiere decir que está funcionando.



Ahora le damos click en el botón que dice obtener datos y nos arroja datos, que son datos de la ruta o url que se estableció en el código del archivo app.js en el fetch.



Después de hacer esto nos vamos a application y buscamos en el menú de la barra izquierda el cache storage y vemos que ahí esta el cache de esta practica le damos click y nos muestra la lista de bucket que esta guardando datos.

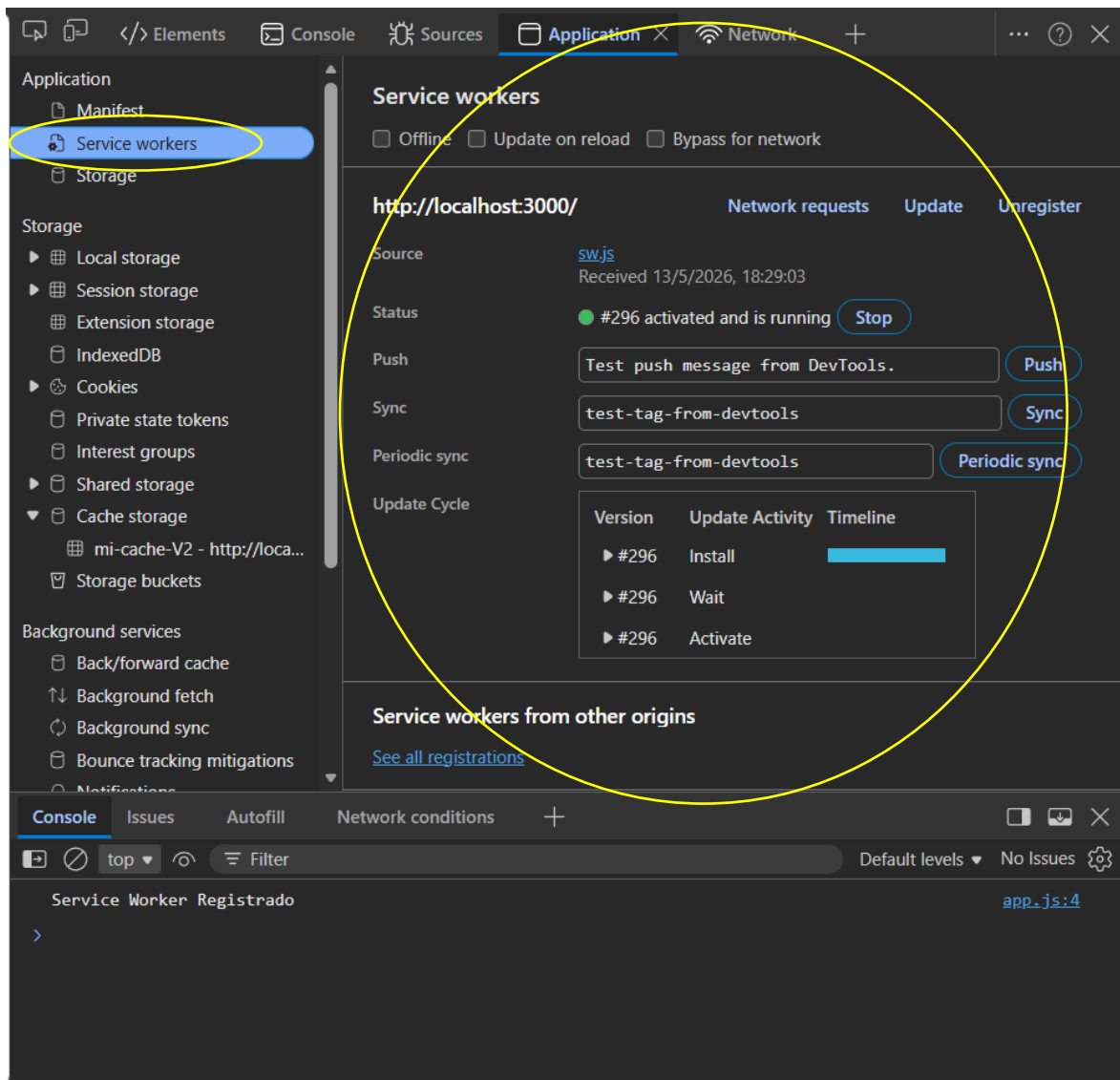


The screenshot shows the Chrome DevTools Application tab. The left sidebar is expanded to the 'Storage' section, where 'Cache storage' is selected. Under 'Cache storage', the entry 'mi-cache-V2 - http://localhost:3000' is highlighted. The main panel shows the 'http://localhost:3000' cache view. It displays a table of cached resources for the 'default' bucket.

#	Name	Respo...	Conte...	Conte...	Time ...	Vary H...
0	/	basic	text/ht...	430	13/5/2...	
1	/app.js	basic	text/ja...	601	13/5/2...	
2	/index.html	basic	text/ht...	430	13/5/2...	

Below the table, it says 'No cache entry selected' and 'Select a cache entry above to preview'. At the bottom, it indicates 'Total entries: 3'.

Despues en el mismo menú izquierdo buscamos el service worker que aparece en la categoría de application, le damos click y podemos ver que nos aparecen las características de este servicio y que está corriendo. Tambien ahí unas casillas que se pueden seleccionar como para usar sin internet con el puro cache guardado, actualizar u correr el servicio o por bypass en la red.



Nos vamos a la pestaña Network y pues podemos ver que esta el localhost y la app.js que están funcionando con red o también se puede sin red. Y indica varias características de estos y el servicio en el que están.

The screenshot shows the Chrome DevTools Network tab. The top bar includes tabs for Elements, Console, Sources, Application, and Network (selected). Below the tabs are various settings like 'No throttling' and 'Preserve log'. The main area displays a table of network requests. A yellow oval highlights the first two rows: 'localhost' and 'app.js'. Below the table, a summary bar shows '2 requests', '0 B transferred', '1.0 kB resources', 'Finish: 9 ms', 'DOMContentLoaded: 16 ms', and 'Load: 17 ms'. At the bottom, the Console tab is active, showing the message 'Service Worker Registrado' with a link to 'app.js:4'.

Name	Status	Type	Initiator	Size	Time	Fulfilled by
localhost	200	document	Other	0 B	3 ms	(ServiceWo...
app.js	200	script	(index):14	0 B	2 ms	(ServiceWo...

2 requests 0 B transferred 1.0 kB resources Finish: 9 ms DOMContentLoaded: 16 ms Load: 17 ms

Console Issues Autofill Network conditions +

Service Worker Registrado [app.js:4](#)

Ya miramos que lo anterior funciono, vamos a irnos a el archivo app.js y agregamos este código ya sea al inicio o al final del código que ya se tiene y guardamos.

```
app.get('/datos', (req, res) =>{  
  res.json({  
    nombre: 'Nayeli',  
    carrera: 'Ingenieria en Sistemas'  
  });  
});
```

Ahora entramos en el archivo sw.js y en las rutas de aquí agregamos la del final que es la que vamos a necesitar para los datos que ingresamos anteriormente.

```
const urlsToCache = [  
  '/',  
  '/index.html',  
  '/app.js',  
  '/datos'  
];
```

Después en el archivo app.js en el fetch donde utilizábamos una ruta de internet para obtener el texto o datos le cambiamos a la ruta de “/datos” para que se utilicen los datos que utilizamos anteriormente.

```
document.getElementById('btn').addEventListener('click', () =>{  
  fetch('/datos')
```

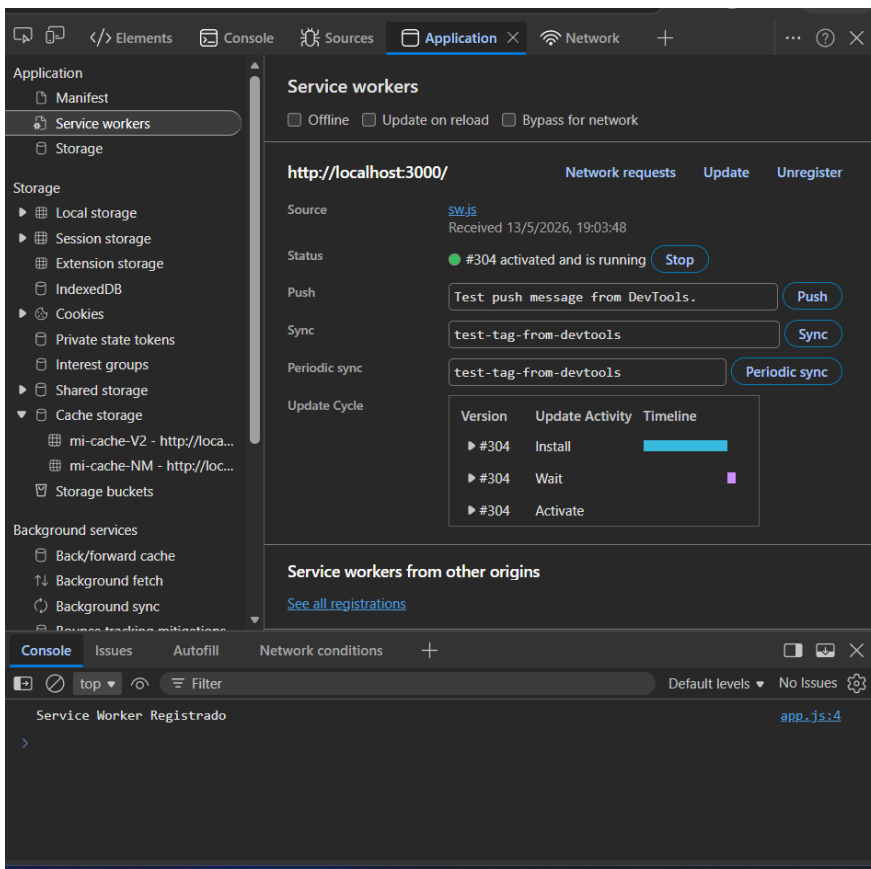
En el inicio del sw.js tenemos el nombre del cache anterior y tenemos que cambiarle el nombre para que nos aparezca el nuevo cache.

```
const CACHE_NAME = 'mi-cache-NM';
```

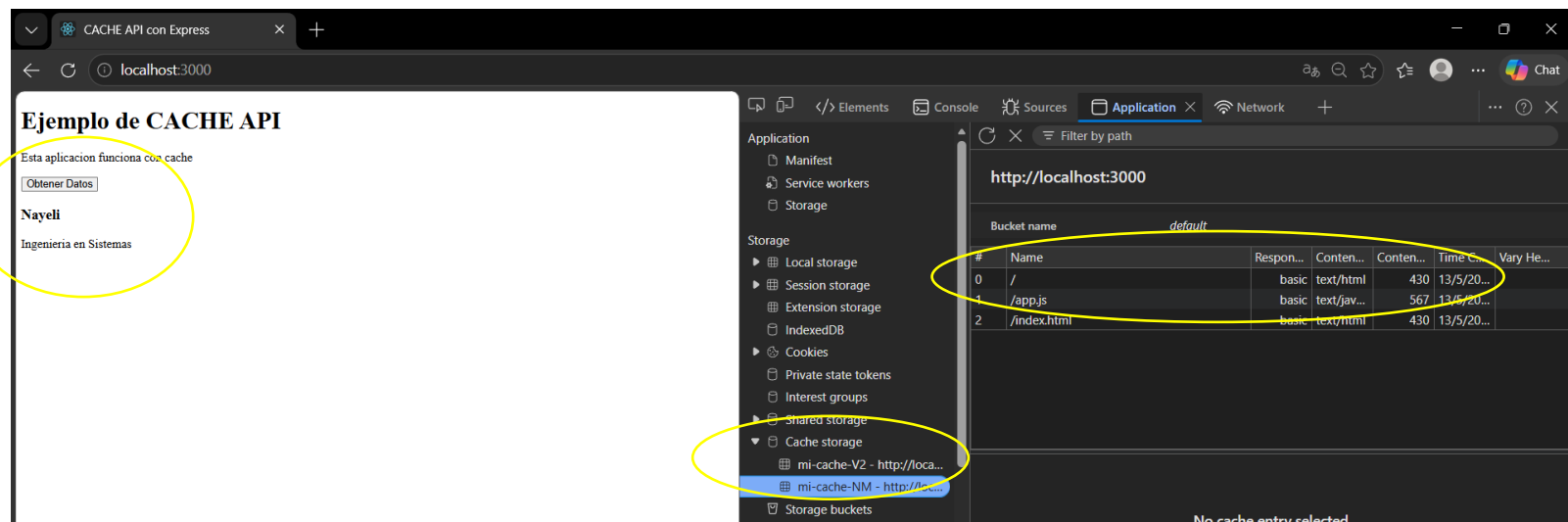
Ahora por último le cambiamos en el código del app.js del fetch donde mandamos llamar los resultados para que se muestren en el HTML, en los valores de los datos ingresamos los que agregamos en el server.js que son nombre y carrera porque si no pues no se van a mostrar.

```
document.getElementById('btn').addEventListener('click', () =>{
  fetch('/datos')
    .then(res => res.json())
    .then(data =>{
      document.getElementById('resultado').innerHTML = `
        <h3>${data.nombre}</h3>
        <p>${data.carrera}</p>
      `;
    })
    .catch(err => console.log(err));
});
```

Aquí volvemos a ver el service worker que esta funcionando, pero tenemos que recargar la pagina para las actualizaciones que se hicieron en el código.



Por último, en el menú lateral izquierdo volvemos a buscar cache storage y podemos ver que aparece el nuevo cache que tenemos y pues los archivos que usan el cache o que guardan los datos. Ahora le damos click en el botón de obtener datos y van a aparecer los que ingresamos en el app.js en la ruta de datos.



Conclusion

En conclusión, esta práctica permitió entender la importancia de la Cache API y los Service Workers en el desarrollo web moderno. Se comprobó cómo es posible mejorar la velocidad de carga y la disponibilidad de una aplicación al utilizar estrategias como "cache primero, red después". Además, se identificaron posibles errores en la implementación, lo cual ayudó a reforzar el aprendizaje y la importancia de escribir código consistente. En general, esta actividad fortaleció los conocimientos sobre aplicaciones web progresivas y el manejo de recursos en caché para optimizar la experiencia del usuario.